

**Министерство науки и высшего образования Российской Федерации**  
**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ**  
**УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**  
**НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ**  
**ИТМО**  
**ITMO University**

**ЛАБОРАТОРНАЯ РАБОТА №2**

**По дисциплине:** Алгоритмы и структуры данных

**Тема работы:** Сложность алгоритмов.

**Обучающиеся:** Казанцев Александр Сергеевич

**Факультет:** Факультет прикладной информатики

**Направление подготовки:** 11.04.02 Инфокоммуникационные технологии и системы связи

**Группа:** К4112с

**Руководитель:** Мусаев Андрей Александрович

Санкт-Петербург, 2026

## **СОДЕРЖАНИЕ**

|                                         | Стр. |
|-----------------------------------------|------|
| <b>ВВЕДЕНИЕ</b>                         | 3    |
| <b>1 ОСНОВНОЕ ЗАДАНИЕ</b>               | 4    |
| 1.1 Задание 1                           | 4    |
| 1.2 Задание 2                           | 5    |
| <b>2 ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ</b>         | 16   |
| <b>ЗАКЛЮЧЕНИЕ</b>                       | 16   |
| <b>СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ</b> | 16   |

## **ВВЕДЕНИЕ**

Целью этой работы является изучение вычислительной сложности программ и освоение оценки времени работы через нотацию  $O$  большое. Мы на практике разберем поведение разных функций при росте объема входных данных и научимся заранее понимать поведение бэкенда под нагрузкой. Для этого мы напишем десять различных программ на языке Go и проведем сравнительный анализ их эффективности.

# 1 ОСНОВНОЕ ЗАДАНИЕ

## 1.1 Задание 1

### Условие:

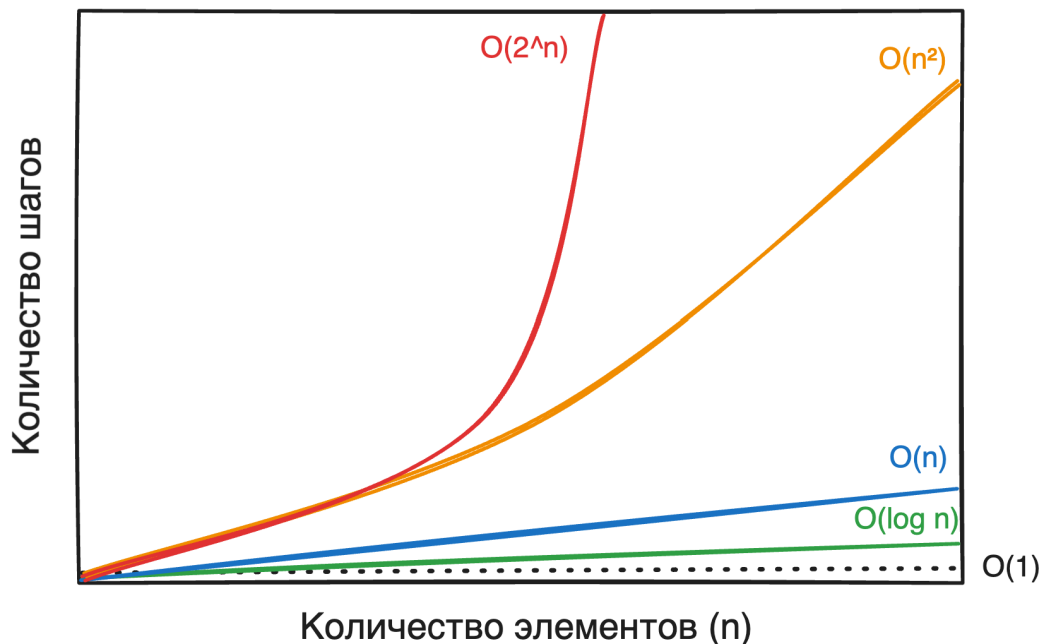
Построить график зависимости между количеством элементов и количеством шагов для алгоритмов со сложностью  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n^2)$ ,  $O(2^n)$ . Сравнить сложность данных алгоритмов.

### Объяснение решения:

Чтобы сравнить скорость алгоритмов, я нарисовал график, где показал, как растет число шагов с увеличением количества элементов.

По линиям сразу видно, что константная сложность  $O(1)$  и логарифмическая  $O(\log n)$  - **наиболее эффективные**, они вообще почти не меняются. Линия  $O(n)$  идет ровно и предсказуемо. А вот  $O(n^2)$  и  $O(2^n)$  моментально улетают в космос. Становится понятно, что на реальном бэкенде с тяжелыми базами данных алгоритмы с такой взрывной сложностью использовать просто нельзя, иначе сервер сразу ляжет.

### График зависимостей (Big O):



## 1.2 Задание 2

### Условие:

Шаг 1: Придумать и написать программы.

- Вам нужно создать 10 отдельных функций или блоков кода.
- Они должны быть сгруппированы в 5 пар (см. Таблицу 1 ст. 1).
- В каждой паре две программы должны иметь одинаковую "скорость" (одинаковую сложность  $O$ ) (см. Таблицу 1 ст. 2), но решать разные задачи или решать одну задачу разными способами (см. Таблицу 1 ст. 3).

Шаг 2: Объяснить свои решения.

- После написания кода вам нужно для каждой программы дать описание, объясняющее, почему в программе именно та сложность, которая от требовалась.

Таблица 1

| № Функции | Сложность $O(n)$ | Аналогия из жизни                                                                 |
|-----------|------------------|-----------------------------------------------------------------------------------|
| 1, 2      | $O(3n)$          | Сделать 3 круга по списку гостей                                                  |
| 3, 4      | $O(n \log n)$    | Найти человека в отсортированном телефонном справочнике                           |
| 5, 6      | $O(n^3)$         | Найти все возможные тройки гостей для фото                                        |
| 7, 8      | $O(n!)$          | Составить все возможные порядки рассадки гостей за столом                         |
| 9, 10     | $O(3 \log n)$    | Угадать загаданное число от 1 до 100, задавая только вопросы "больше или меньше"? |

### Объяснение решения:

Сейчас в ИТМО большая очередь на общежитие и много кто из магистров до сих пор не заселен. Я сам стою в этом списке 600+ по счету и жду своей очереди поэтому решил реализовать все функции вокруг темы распределения мест в общагах и проверки списков студентов. 10 функций я разбил по парам чтобы показать как разные задачи на бэкенде могут иметь одинаковую асимптотическую скорость выполнения.

1) В первой паре со сложностью  $O(3n)$  код делает три отдельных круга по массиву студентов один за другим. Первая функция считает общую статистику по тем, кто ждет места, а вторая просто чистит текстовые имена студентов от случайных лишних пробелов на концах строк. Математически здесь нет вложенности, поэтому шаги суммируются как  $n$  плюс  $n$  плюс  $n$ , что дает **три  $n$** . По правилам нотации константы отбрасываются, и получается линейная сложность, но в коде мы явно выполняем три отдельных прохода.

## Код 1 пары на Golang:

```
main.go
lab-02-complexity > 01_three_loops > main.go
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 type Student struct {
9     Name      string
10    IsMagistr  bool
11    InQueue    bool // true - ждёт общагу, false - уже заселился
12    QueueNumber int // номер в очереди, для заселенных 0
13 }
14
15 // getQueueStats - сбор аналитики по очередникам
16 func getQueueStats(students []Student) (int, int, int) {
17     magistrsInQueue := 0
18     totalInQueue := 0
19     maxQueueNumber := 0
20
21     for _, s := range students {
22         if s.IsMagistr && s.InQueue {
23             magistrsInQueue++
24         }
25     }
26
27     for _, s := range students {
28         if s.InQueue {
29             totalInQueue++
30         }
31     }
32
33     for _, s := range students {
34         if s.InQueue && s.QueueNumber > maxQueueNumber {
35             maxQueueNumber = s.QueueNumber
36         }
37     }
38
39     return magistrsInQueue, totalInQueue, maxQueueNumber
40 }
41
42 // cleanStudentNames - очистка текстовых данных
43 func cleanStudentNames(names []string) []string {
44     for i := range names {
45         names[i] = strings.TrimSpace(names[i])
46     }
47
48     for i := range names {
49         names[i] = strings.ToLower(names[i])
50     }
51
52     var validNames []string
53
54     for i := range names {
55         if names[i] != "" {
56             validNames = append(validNames, names[i])
57         }
58     }
59
60     return validNames
61 }
```

```

63 func main() {
64     groupData := []Student{
65         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
66             150},
67         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
68             QueueNumber: 0},
69         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
70             QueueNumber: 342},
71         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
72             612},
73         {Name: "Кенжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
74             89},
75         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
76             540},
77         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
78             0},
79         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
80             QueueNumber: 221},
81         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
82             431},
83         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
84             QueueNumber: 641}, // Это ты, твоя реальная позиция
85         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
86             112},
87         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
88             299},
89         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
90             QueueNumber: 0},
91     }
92
93     magInQueue, totalInQueue, maxNum := getQueueStats(groupData)
94
95     fmt.Printf("Магистров в очереди: %d\n", magInQueue)
96     fmt.Printf("Всего студентов в очереди: %d\n", totalInQueue)
97     fmt.Printf("Максимальный номер в очереди потока: %d\n\n", maxNum)
98
99     rawDorms := []string{"Студгородок Вязьма ", "Общежитие №2 Ленсовета ",
100         "", "Альпы "}
101
102     cleanedDorms := cleanStudentNames(rawDorms)
103
104     fmt.Println("Результат работы очистки названий:")
105     for _, name := range cleanedDorms {
106         fmt.Println("-", name)
107     }
108 }

```

```

→ 01_three_loops go run main.go
Магистров в очереди: 8
Всего студентов в очереди: 10
Максимальный номер в очереди потока: 641

```

Результат работы очистки названий:

```

- студгородок вязьма
- общежитие №2 ленсовета
- альпы

```

```

→ 01_three_loops |

```

2) Во второй паре  $O(n \log n)$  мы используем сортировку и быстрый поиск. Одна функция расставляет студентов по их приоритету в очереди через алгоритм быстрой сортировки, который на каждом этапе делит массив пополам и тратит на это в среднем  $n \log n$  шагов. Вторая функция берет исходный массив из  $n$  студентов и для каждого из них запускает бинарный поиск по отсортированному списку приказов. Так как один бинарный поиск занимает логарифмическое время, а мы вызываем его в цикле  $n$  раз, общая скорость получается ровно  $n \log n$ .

Код 2 пары на Golang:

```
main.go x
lab-02-complexity > 02_n_log_n > main.go
1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // quickSortStudents - сортировка студентов по номеру в очереди
13 func quickSortStudents(students []Student) []Student {
14     if len(students) < 2 {
15         return students
16     }
17
18     pivot := students[0]
19
20     var less []Student
21     var greater []Student
22
23     for _, s := range students[1:] {
24         if s.QueueNumber <= pivot.QueueNumber {
25             less = append(less, s)
26         } else {
27             greater = append(greater, s)
28         }
29     }
30
31     result := append(quickSortStudents(less), pivot)
32     return append(result, quickSortStudents(greater)...)
33 }
34
35 // findSettledStudents - поиск студентов в утвержденных приказах ИТМО
36 func findSettledStudents(students []Student, activeOrders []int) []string {
37     var settled []string
38
39     for _, s := range students {
40         low := 0
41         high := len(activeOrders) - 1
42         found := false
43         for low <= high {
44             mid := low + (high-low)/2
45
46             if activeOrders[mid] == s.QueueNumber {
47                 found = true
48                 break
49             }
50
51             if activeOrders[mid] < s.QueueNumber {
52                 low = mid + 1
53             } else {
54                 high = mid - 1
55             }
56         }
57
58         if found && s.QueueNumber != 0 {
59             settled = append(settled, s.Name)
60         }
61     }
62
63     return settled
64 }
65
```

```

66 func main() {
67     groupData := []Student{
68         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
150},
69         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
QueueNumber: 0},
70         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
QueueNumber: 342},
71         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
612},
72         {Name: "Кенжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
89},
73         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
540},
74         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
0},
75         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
QueueNumber: 221},
76         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
431},
77         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
QueueNumber: 641},
78         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
112},
79         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
299},
80         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
QueueNumber: 0},
81     }

82     sorted := quickSortStudents(groupData)
83     fmt.Println("Студенты, отсортированные по номеру в очереди:")
84     for _, s := range sorted {
85         if s.InQueue {
86             fmt.Printf("- %s: №%d\n", s.Name, s.QueueNumber)
87         }
88     }
89     fmt.Println()
90     ordersInViazma := []int{89, 112, 221, 299, 641}
91     luckyStudents := findSettledStudents(groupData, ordersInViazma)
92     fmt.Println("Студенты, чьи номера найдены в приказах на заселение в
Вязьму:")
93     for _, name := range luckyStudents {
94         fmt.Println("-", name)
95     }
96 }
97
100 }

```

```

02_n_log_n go run main.go
Студенты, отсортированные по номеру в очереди:
- Кенжаев Рахим: №89
- Юркин Александр: №112
- Егорова Мария: №150
- Бабушкин Александр: №221
- Панкратова Анна: №299
- Костюченко Тимофей: №342
- Якунин Андрей: №431
- Даньшин Семён: №540
- Портнова Ксения: №612
- Казанцев Александр: №641

Студенты, чьи номера найдены в приказах на заселение в Вязьму:
- Кенжаев Рахим
- Бабушкин Александр
- Казанцев Александр
- Юркин Александр
- Панкратова Анна
02_n_log_n

```

3) В третьей паре  $O(n^3)$  у нас кубическая сложность за счет трех вложенных циклов. Первая программа подбирает уникальные тройки студентов для совместного заселения в трехместные комнаты в Вязьме, а вторая собирает отчет по свободным местам, обходя жилые блоки, комнаты и спальные места. На каждый шаг первого цикла полностью запускается второй а внутри него третий поэтому общее число проверок растет пропорционально кубу от количества элементов.

### Код 3 пары на Golang:

```
lab-02-complexity > 03_cubic_loops > main.go
1  package main
2
3  import "fmt"
4
5  type Student struct {
6      Name      string
7      IsMale    bool // true - парень, false - девушка
8      IsMagistr bool
9      InQueue   bool
10     QueueNumber int
11 }
12
13 type RoomBlock struct {
14     BlockNumber int
15     Rooms      [][]bool // true - кровать свободна
16 }
17
18 // findRoomTriplets - подбор уникальных троек студентов для трехместной комнаты
19 // в комнату могут заселяться студенты только одного пола!
20 func findRoomTriplets(students []Student, maxCombineQueue int) [][]string {
21     var triplets [][]string
22     n := len(students)
23
24     for i := range n {
25         for j := i + 1; j < n; j++ {
26             for k := j + 1; k < n; k++ {
27                 if students[i].InQueue && students[j].InQueue && students[k].InQueue {
28                     allMales := students[i].IsMale && students[j].IsMale && students[k].IsMale
29                     allFemales := !students[i].IsMale && !students[j].IsMale && !students[k].IsMale
30
31                     if allMales || allFemales {
32                         sumQueue := students[i].QueueNumber + students[j].QueueNumber + students[k].QueueNumber
33                         if sumQueue <= maxCombineQueue {
34                             triplets = append(triplets, []string{students[i].Name, students[j].Name, students[k].Name})
35                         }
36                     }
37                 }
38             }
39         }
40     }
41
42     return triplets
43 }
44
45 // findFreeBeds - поиск свободных коек
46 func findFreeBeds(blocks []RoomBlock) []string {
47     var freeBedsReport []string
48
49     for _, block := range blocks {
50         for roomIdx, room := range block.Rooms {
51             for bedIdx, isFree := range room {
52                 if isFree {
53                     report := fmt.Sprintf("Блок %d, Комната %d, Кровать %d",
54                         block.BlockNumber, roomIdx+1, bedIdx+1)
55                     freeBedsReport = append(freeBedsReport, report)
56                 }
57             }
58         }
59     }
60
61     return freeBedsReport
62 }
```

```

63 func main() {
64     groupData := []Student{
65         {Name: "Егорова Мария", IsMale: false, IsMagistr: true, InQueue: true,
66           QueueNumber: 150},
67         {Name: "Фролова Кристина", IsMale: false, IsMagistr: true, InQueue:
68           false, QueueNumber: 0},
69         {Name: "Костюченко Тимофей", IsMale: true, IsMagistr: false, InQueue:
70           true, QueueNumber: 342},
71         {Name: "Портнова Ксения", IsMale: false, IsMagistr: true, InQueue:
72           true, QueueNumber: 612},
73         {Name: "Кенжаев Рахим", IsMale: true, IsMagistr: true, InQueue: true,
74           QueueNumber: 89},
75         {Name: "Даньшин Семён", IsMale: true, IsMagistr: true, InQueue: true,
76           QueueNumber: 540},
77         {Name: "Шихайло Сергей", IsMale: true, IsMagistr: false, InQueue:
78           false, QueueNumber: 0},
79         {Name: "Бабушкин Александр", IsMale: true, IsMagistr: true, InQueue:
80           true, QueueNumber: 221},
81         {Name: "Якунин Андрей", IsMale: true, IsMagistr: true, InQueue: true,
82           QueueNumber: 431},
83         {Name: "Казанцев Александр", IsMale: true, IsMagistr: true, InQueue:
84           true, QueueNumber: 641},
85         {Name: "Юркин Александр", IsMale: true, IsMagistr: true, InQueue: true,
86           QueueNumber: 112},
87         {Name: "Панкратова Анна", IsMale: false, IsMagistr: false, InQueue:
88           true, QueueNumber: 299},
89         {Name: "Колесникова Лариса", IsMale: false, IsMagistr: true, InQueue:
90           false, QueueNumber: 0},
91     }
92 }
93
94 fmt.Println("Подобраны тройки студентов для Вязьмы:")
95 triplets := findRoomTriplets(groupData, 500)
96 for _, t := range triplets {
97     fmt.Printf("- %s, %s, %s\n", t[0], t[1], t[2])
98 }
99 fmt.Println()
100
101 viazmaBlocks := []RoomBlock{
102     {
103         BlockNumber: 4,
104         Rooms: [][]bool{
105             {false, false, true},
106             {false, false, false},
107         },
108     },
109     {
110         BlockNumber: 5,
111         Rooms: [][]bool{
112             {true, false, true},
113         },
114     },
115 }
116
117 fmt.Println("Свободные места обнаружены:")
118 freeBeds := findFreeBeds(viazmaBlocks)
119 for _, bed := range freeBeds {
120     fmt.Println("-", bed)
121 }
122 }
123
124 → 03_cubic_loops go run main.go
Подобраны тройки студентов для Вязьмы:
- Кенжаев Рахим, Бабушкин Александр, Юркин Александр

Свободные места обнаружены:
- Блок 4, Комната 1, Кровать 3
- Блок 5, Комната 1, Кровать 1
- Блок 5, Комната 1, Кровать 3
→ 03_cubic_loops

```

4) Четвёртая пара - сложность  $O(n!)$ , данная сложность представляет собой жёсткий рекурсивный перебор всех возможных комбинаций со взрывным ростом шагов. Одна функция составляет порядок обхода комнат комендантом в корпусе на Ленсовета, а вторая ищет **самый короткий маршрут** для студента между общежитием и учебными корпусами через метро. Чтобы найти этот оптимальный **путь**, коду приходится перебирать абсолютно все перестановки точек, что даёт факториальную сложность.

**Код 4 пары на Golang:**

```
lab-02-complexity > 04_factorial > main.go
1 package main
2
3 import "fmt"
4
5 // findRoomPaths - находит все варианты порядка обхода комнат
6 func findRoomPaths(rooms []string, start int, result *[][]string) {
7     if start == len(rooms) {
8         temp := make([]string, len(rooms))
9         copy(temp, rooms)
10        *result = append(*result, temp)
11        return
12    }
13
14    for i := start; i < len(rooms); i++ {
15        rooms[start], rooms[i] = rooms[i], rooms[start]
16        findRoomPaths(rooms, start+1, result)
17        rooms[start], rooms[i] = rooms[i], rooms[start]
18    }
19 }
20
21 // findShortestRoute - поиск кратчайшего пути
22 func findShortestRoute(
23     locations []string,
24     startIdx int,
25     currentRoute []string,
26     visited []bool,
27     matrix [][]int,
28     currentDist int,
29     bestDist *int,
30     finalRoute *[]string,
31 ) {
32     if len(currentRoute) == len(locations) {
33         finalDist := currentDist + matrix[startIdx][0]
34         if finalDist < *bestDist {
35             *bestDist = finalDist
36             *finalRoute = append([]string(nil), currentRoute...)
37             *finalRoute = append(*finalRoute, locations[0])
38         }
39         return
40     }
41
42     for i := range locations {
43         if !visited[i] {
44             visited[i] = true
45             findShortestRoute(locations, i, append(currentRoute, locations[i]),
46                 visited, matrix, currentDist+matrix[startIdx][i], bestDist,
47                 finalRoute)
48             visited[i] = false
49         }
50     }
51 }
52
53 func main() {
54     roomsList := []string{"Комната 201", "Комната 202", "Комната 203"}
55     var options [][]string
56     findRoomPaths(roomsList, 0, &options)
57
58     fmt.Printf("Всего вариантов обхода для %d комнат: %d\n", len(roomsList), len
59     (options))
60     fmt.Println("Варианты маршрутов на Ленсовета:")
61     for _, p := range options {
62         fmt.Printf("- %s -> %s -> %s\n", p[0], p[1], p[2])
63     }
64     fmt.Println()
65
66     itmoLocations := []string{"Обшара Вавиловых", "Корпус Кронверкский",
67     "Корпус Ломоносова", "Корпус Биржевая"}
68
69     distanceMatrix := [][]int{
70         {0, 45, 50, 55},
71         {45, 0, 25, 20},
72         {50, 25, 0, 30},
73         {55, 20, 30, 0},
74     }
75 }
```

```

71
72     visited := make([]bool, len(itmoLocations))
73     visited[0] = true
74
75     minMinutes := 999999
76     var pathResult []string
77
78     findShortestRoute(itmoLocations, 0, []string{itmoLocations[0]}, visited,
79         distanceMatrix, 0, &minMinutes, &pathResult)
80
81     fmt.Printf("Самый короткий путь через метро: %d минут\n", minMinutes)
82     fmt.Println("Порядок движения:")
83     for i, loc := range pathResult {
84         if i > 0 {
85             fmt.Print(" -> ")
86         }
87         fmt.Print(loc)
88     }
89     fmt.Println()
90 }

```

→ 04\_factorial go run main.go  
 Всего вариантов обхода для 3 комнат: 6  
 Варианты маршрутов на Ленсовета:  
 - Комната 201 -> Комната 202 -> Комната 203  
 - Комната 201 -> Комната 203 -> Комната 202  
 - Комната 202 -> Комната 201 -> Комната 203  
 - Комната 202 -> Комната 203 -> Комната 201  
 - Комната 203 -> Комната 202 -> Комната 201  
 - Комната 203 -> Комната 201 -> Комната 202  
  
 Самый короткий путь через метро: 145 минут  
 Порядок движения:  
 Общага Вавиловых -> Корпус Кронверкский -> Корпус Биржевая -> Корпус Ломоносова -> Общага Вавиловых  
 → 04\_factorial

5) В последней паре  $O(3 \log n)$  делаем три бинарных поиска подряд. Первая функция ищет ID студента сразу в трех разных базах по корпусам, а вторая проверяет свободные места на конкретные даты заселения по трем разным адресам. Каждый бинарный поиск делит диапазон данных пополам и выполняется за логарифм, а так как они идут строго друг за другом без вложенности, их сложности складываются и получается три логарифма  $n$ .

**Код 5 пары на Golang:**

```
lab-02-complexity > 05_three_log > main.go
1 package main
2
3 import "fmt"
4
5 var studentDB = map[int]string{
6     368088: "Даньшин Семён",
7     506428: "Егорова Мария",
8     563346: "Казанцев Александр",
9     354511: "Кенжаев Рахим",
10    564670: "Колесникова Лариса",
11    563558: "Костюченко Тимофей",
12    563102: "Панкратова Анна",
13    367496: "Портнова Ксения",
14    368978: "Фролова Кристина",
15    182717: "Шихайло Сергей",
16    367662: "Юркин Александр",
17    369105: "Якунин Андрей",
18 }
19
20 // findStudentID - поиск ID студента по трем базам общежитий
21 func findStudentID(viazma, lensoveta, msg []int, targetID int) string {
22     low, high := 0, len(viazma)-1
23     for low <= high {
24         mid := low + (high-low)/2
25         if viazma[mid] == targetID {
26             return fmt.Sprintf("%s в списках Вязьмы", studentDB[targetID])
27         }
28         if viazma[mid] < targetID {
29             low = mid + 1
30         } else {
31             high = mid - 1
32         }
33     }
34
35     low, high = 0, len(lensoveta)-1
36     for low <= high {
37         mid := low + (high-low)/2
38         if lensoveta[mid] == targetID {
39             return fmt.Sprintf("%s в списках на Ленсовета", studentDB[targetID])
40         }
41         if lensoveta[mid] < targetID {
42             low = mid + 1
43         } else {
44             high = mid - 1
45         }
46     }
47
48     low, high = 0, len(msg)-1
49     for low <= high {
50         mid := low + (high-low)/2
51         if msg[mid] == targetID {
52             return fmt.Sprintf("%s в списках Межвузовского студгородка",
53                 studentDB[targetID])
54         }
55         if msg[mid] < targetID {
56             low = mid + 1
57         } else {
58             high = mid - 1
59         }
60     }
61
62     return "Студент нигде не найден!"
63 }
64
65 // findFreeDate - поиск свободной даты для заселения
66 func findFreeDate(schedule1, schedule2, schedule3 []int, targetDate int) string
67 {
68     low, high := 0, len(schedule1)-1
69     for low <= high {
70         mid := low + (high-low)/2
71         if schedule1[mid] == targetDate {
72             return "свободный слот найден в первом графике"
```

```

72         if schedule1[mid] < targetDate {
73             low = mid + 1
74         } else {
75             high = mid - 1
76         }
77     }
78
79     low, high = 0, len(schedule2)-1
80     for low <= high {
81         mid := low + (high-low)/2
82         if schedule2[mid] == targetDate {
83             return "свободный слот найден во втором графике"
84         }
85         if schedule2[mid] < targetDate {
86             low = mid + 1
87         } else {
88             high = mid - 1
89         }
90     }
91
92     low, high = 0, len(schedule3)-1
93     for low <= high {
94         mid := low + (high-low)/2
95         if schedule3[mid] == targetDate {
96             return "свободный слот найден в третьем графике"
97         }
98         if schedule3[mid] < targetDate {
99             low = mid + 1
100         } else {
101             high = mid - 1
102         }
103     }
104
105     return "все слоты уже заняты!"
106 }
107
108 func main() {
109     // Массивы отсортированы для бинарного поиска
110     viazmaIDs := []int{367496, 368978, 563346, 563558}
111     lensovetaIDs := []int{182717, 354511, 367662, 368088}
112     msgIDs := []int{369105, 506428, 563102, 564670}
113
114     myID := 563346
115     resultDorm := findStudentID(viazmaIDs, lensovetaIDs, msgIDs, myID)
116     fmt.Printf("Поиск студента с ID %d: %s\n", myID, resultDorm)
117
118     mariaID := 506428
119     resultGirl := findStudentID(viazmaIDs, lensovetaIDs, msgIDs, mariaID)
120     fmt.Printf("Поиск студента с ID %d: %s\n\n", mariaID, resultGirl)
121
122     dates1 := []int{1, 3, 5, 7}
123     dates2 := []int{10, 12, 14, 15}
124     dates3 := []int{20, 22, 25, 28}
125
126     myDay := 27
127     mySlot := findFreeDate(dates1, dates2, dates3, myDay)
128     fmt.Printf("Поиск свободного слота для Казанцева Александра на %d октября: %s\n", myDay, mySlot)
129
130     girlDay := 14
131     girlSlot := findFreeDate(dates1, dates2, dates3, girlDay)
132     fmt.Printf("Поиск свободного слота для Егоровой Марии на %d октября: %s\n", girlDay, girlSlot)
133 }
134
135 → 05_three_log go run main.go
Поиск студента с ID 563346: Казанцев Александр в списках Вязьмы
Поиск студента с ID 506428: Егорова Мария в списках Межвузовского студгородка

Поиск свободного слота для Казанцева Александра на 27 октября: все слоты уже заняты!
Поиск свободного слота для Егоровой Марии на 14 октября: свободный слот найден во втором графике
→ 05_three_log

```

Этот последовательный разбор наглядно показывает разницу в скоростях работы алгоритмов при обработке базы данных студентов ИТМО. Такой анализ позволяет сразу видеть неэффективные функции в кодовой базе и заменять их на быстрые логарифмические решения.

## 2 ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ

### Условие:

Опишите, какая сложность была у вашего решения второго задания из первой работы.

### Объяснение решения:

Во второй задаче первой лабораторной работы мной была реализована экспертная система для угадывания студента по его анкете. Алгоритм работает в два независимых этапа:

1. **Консольный опрос пользователя:** Программа последовательно задаёт фиксированное количество вопросов (7 штук), считывая ответы через **bufio.NewReader**. Так как число вопросов строго ограничено и не зависит от общего размера учебной группы, этот этап имеет константную сложность  **$O(1)$** .
2. После опроса запускается один цикл **for range**. Программа по очереди берёт каждого из **n** студентов группы и проверяет совпадение параметров. Этот проход напрямую зависит от размера группы, что даёт сложность  **$O(n)$** .

Поскольку эти этапы выполняются последовательно, их сложности складываются:  $O(1) + O(n)$ . По правилам асимптотического анализа константы отбрасываются, поэтому итоговая сложность всего алгоритма является линейной -  **$O(n)$** .

## ЗАКЛЮЧЕНИЕ

Данная лабораторная работа помогла мне на практике освоить оценку вычислительной сложности программ и изучить поведение различных алгоритмов, при постоянном росте объёма входных данных **n** **стремиться к бесконечности**. Написание десяти программ на языке Go и их группировка по парам наглядно доказали, что совершенно разные задачи могут иметь одинаковую асимптотическую скорость выполнения.

Полученные навыки анализа  $O$  большого позволяют ещё на этапе проектирования замечать узкие места в кодовой базе. Это помогает вовремя заменять неэффективные кубические или факториальные циклы на быстрые логарифмические, гарантируя стабильную работу высоконагруженных систем.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бхаргава А. Грокаем алгоритмы. Второе издание, 2026.
2. Документация по языку программирования Go, ссылка: <https://go.dev/doc/>
3. Лекция 2: Оценка сложности алгоритмов, Университет ИТМО